Verslag Geavanceerde Computerarchitectuur

Ybe Vandamme, Jarno De Jonckheere

2024



1 Labo 1: Getting Acquainted with GPU Programming

In dit labo was het de bedoeling om ons kennis te laten maken met GPU programmering. GPU programmering zorgt ervoor dat threads parallel kunnen worden uitgevoerd. De focus was dus om de structuur van het programmeren onder te knie te krijgen.
In dit labo werd er ons gevraagd om een rij in omgekeerde volgorde te bekomen. Dit kon op verschillende manieren. We konden dit doen a.d.h.v. 1 block met daarin 9 threads. Hierin zou het dus lijken op CPU programmering waarin elke thread om de beurt werd uitgevoerd. Maar we konden dit ook doen door 9 blocks met daarin elk 1 thread. Hierbij werd elke thread dus op een aparte Core parallel uitgevoerd. Hierin hebben we dus kunnen experimenteren en kijken wat de effecten zijn. Hieronder zie je een foto van de uitgevoerde code waarbij ook een timestamp werd gegeven om de snelheid waar te nemen van de verschillende testopstellingen.

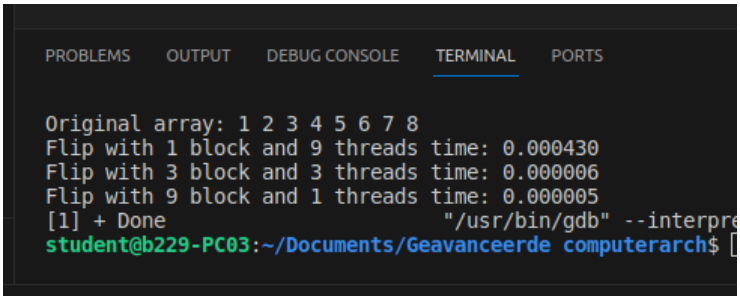


Figure 1: Output array flip

Hierop zien we dus dat het zeker een effect heeft om dingen parallel te laten lopen. Er is een gigantische tijdswinst door met parallelle threads te werken.

2 Labo 2: Race conditions

In dit labo was het de bedoeling om verschillende max-detectie algoritmes te implementeren en deze met elkaar te vergelijken. De drie verchillende implementaties die terug te vinden zijn in ons experiment zijn:

- Max via een for-loop op de CPU
- Max via atomic operations op de GPU
- Max via een reduction-algoritme op de GPU

In afbeelding 2 ziet u de resultaten van ons experiment. Merkwaardig is hierbij dat het reductie-algoritme het slechtst presteert. Dit is te verklaren door het feit dat de executietijd van een reductie-algoritme sterk afhankelijk is van de implementatie. Verder is het logisch dat het algoritme met atomic operations slechter presteert dan het algoritme op de CPU. Dit is te verklaren door het feit dat hier slechts één thread per keer toegang krijgt tot de array bijkomend met het feit dat de CPU geoptimaliseerd is voor sequentiële operaties, terwijl de GPU geoptimaliseerd is voor parallelle operaties. ten laatste kan men ook zeggen dat de extra overhead gecreëerd door het afsluiten/toekennen van toegang tot de array ook zorgt voor extra vertragingen.

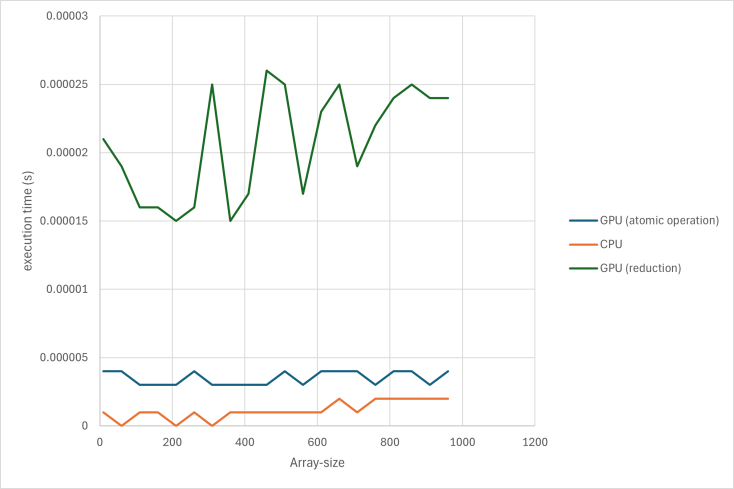


Figure 2: Resultaten experiment

Indien er minder threads voorzien worden dan elementen in de input-array dan zou onze kernel, zoals deze nu geschreven is, niet alle elementen processen.

2.1 Vragen

Q: **Is het resultaat correct?**

Onze resultaten waren bij alle drie de manieren hetzelfde dus gaan we ervan uit dat deze juist zijn. Dit kun je zien in de OUPUT file die is meegeleverd.

Q: **Hoe veranderd de tijd in functie van de input grootte?**

Zie figuur 2.

Q: **Wat gebeurt er als de input groter is dan 2048 (hoe zorg je ervoor dat de correcte berekening wordt gedaan en is er een extra stap nodig)?**

We kunnen de functie atomic max gebruiken. Deze splits de array in blokken en haalt daaruit het maximum. Een andere manier is om de reductie te gebruiken om steeds kleiner te gaan.

Q: **Stel je voor dat je de kernel aanroept met veel minder threads dan er data is. Hoe zorg je ervoor dat alle data wordt geprocessed.**

We kunnen de rij splitsen om maximum aantal data te kunnen processen. Hierachter kunnen we die functie opnieuw oproepen om uit alle maximum waardes het maximum te halen.

Q: **Hoe zou je een 2D rij oplossen?**

Je zou kunnen het maximum bepalen per rij of kolom en die waardes in een nieuwe array steken. Deze nieuwe array zou dan een 1D array zijn en zouden we dus met die voorbeelden hierboven kunnen verder verwerken.